



Faculty of Engineering and Technology
Electrical and Computer Engineering Department Computer
Networks Laboratory ENCS4130

Exp#6 – Access List

Prepared by: Aya Dahbour 1201738

Instructor: Sameh Awad

Teacher assistant: Eng. Ahed Mafarjeh

Section: 8

June – 2018

Abstract

This hands-on experiment delves into the practical implementation of Access Control Lists (ACLs) using routers, aiming to bolster network security. The main goals involve configuring both Standard and Extended ACLs, mastering essential commands, and understanding the distinctions between these ACL types. Employing Cisco Packet Tracer, participants engage in scenarios to selectively permit or deny specific traffic flows, contributing to a comprehensive grasp of ACL configurations. The experiment's deliverables include practical skills in fortifying network security through effective ACL usage.

Contents

Abstract	I
Acronyms and Abbreviations.....	III
List of Figures	IV
List of Tables.....	V
Introduction	6
1.1. Named and Numbered ACL.....	6
1.2. Wildcard Masks	6
1.3. Standard ACL and Extended ACL.....	6
1.4. The Implied "Deny All Traffic" Criteria Statement	6
1.5 Configuring ACL	7
Procedure and Discussion	8
• Building the topology	8
• Configure the IPs	8
• Networks IPS	9
2.1. Configuring OSPF Routing.....	9
2.2. Configuring Standard Access List	9
2.3. Configuring Extended Access List.....	11
Results.....	14
3.1. Configuring Standard Access List	14
3.2. Configuring Extended Access List.....	15
Conclusion	18
References	19

Acronyms and Abbreviations

OSPF	Open Shortest Path First
ACLs	Access Control Lists
HTTP	Hypertext Transfer Protocol
Telnet	Telecommunication Network

List of Figures

Figure 2.1: The Topology	8
Figure 2.2: PC0 IPs	8
Figure 2.1: Configuration OSPF Router1	9
Figure 2.2.1: Prevent PC0 to access 192.38.20.0 network command	10
Figure 2.2.2: permit PC0 to access 192.38.20.0 network command	10
Figure 2.2.3: deny network 192.38.10.0 to access network 192.38.20.0 command	11
Figure 2.2.4: Prevent PC0 to access network 192.38.30.0 command	11
Figure 2.3.2: Allow PC0 to access PC2 all other traffic is denied(command).....	12
Figure 2.3.3: deny PC0 to make HTTP request to this server command	12
Figure 2.3.4: Prevent PC0 from accessing PC4 command.....	12
Figure 2.3.5: deny all the host from making telnet with interface se2/0 of Router1 expect PC0(command)	13
Figure 3.1.1: Prevent PC0 to access 192.38.20.0 network output.....	14
Figure 3.1.2: permit PC0 to access 192.38.20.0 network output	14
Figure 3.1.3: deny network 192.38.10.0 to access network 192.38.20.0 output	15
Figure 3.1.4: Prevent PC0 to access network 192.38.30.0 output.....	15
Figure 3.2.1: Prevent PC0 from accessing PC2 output	15
Figure 3.2.2: Allow PC0 to access PC2 all other traffic is denied(output)	16
Figure 3.2.3: deny PC0 to make HTTP request to this server output.....	16
Figure 3.2.4: Prevent PC0 from accessing PC4 output	17
Figure 3.2.5: deny all the host from making telnet with interface se2/0 of Router1 expect PC0(output)...	17

List of Tables

Table 2.1: Networks IPS	9
-------------------------------	---

1. Introduction

Access Control Lists (ACLs) function as the network's traffic regulators. They consist of sets of guidelines or directives that govern the movement of traffic within and outside a network. Every directive, referred to as an access control entry, specifies the permissible traffic types (such as data categories or source/destination addresses) and those that are prohibited. Frequently, these are simply referred to as access lists or ACLs. Enhancing network security stands as a primary motivation for implementing ACLs. [1]

1.1. Named and Numbered ACL

Access Control Lists (ACLs) can be distinguished through two main approaches. The initial method entails providing them with descriptive names, for instance, "CompanyABC_DataTraffic," serving as convenient labels akin to the nomenclature of streets, like "Main Street." The second approach involves assigning numerical identifiers such as "10" or "101" to the ACLs, analogous to the numeric designations of streets, such as "5th Avenue." These identification methods are frequently utilized on networking devices crafted by Cisco, a distinguished company renowned for manufacturing a diverse array of network hardware. [2]

1.2. Wildcard Masks

When we talk about masks in the context of ACLs, we are talking about something called a wildcard mask. These masks help us specify what should be allowed and what should be denied. Think of it like a stencil that only lets certain things through. In the case of ACLs, these 'stencils' or masks help us to decide which parts of an IP address to look at when deciding whether to let traffic through.

1.3. Standard ACL and Extended ACL

ACLs come in two main types: standard and extended.

1.3.1 Standard ACL:

A standard ACL is like a basic filter. It looks at where the traffic is coming from (the source IP address) and based on that, decides whether to allow it or not. It's a bit like a bouncer at a club who only lets in people from a specific town.

1.3.2 Standard ACL:

An extended ACL is a bit more sophisticated. It not only checks where the traffic is coming from but also where it's going to (the destination IP address). It also looks at what kind of data is being sent (the protocol field) and even checks the port number. This would be like a bouncer who checks where you live, where you're going, what you're carrying, and what door you're using.

1.4. The Implied "Deny All Traffic" Criteria Statement

All ACLs have an unspoken rule at the end that says 'deny all traffic'. This means if none of the other rules in the ACL apply to the traffic, it will be blocked by default. This is a safety measure to make sure that only traffic that has been explicitly allowed can get through.

1.5 Configuring ACL

Setting up an ACL involves several steps, such as defining what to filter, naming or numbering the ACL, and defining the rules. After that, the ACL has to be attached to a particular part of the network (an interface) for it to start working.

1.5.1 Standard ACL Configuration:

Configuring a standard ACL involves writing commands that specify which traffic to allow or deny. For instance, if you wanted to block traffic from a particular network, you'd write a command like 'access-list 10 deny 192.X.0.0 0.0.0.255'. After the ACL is set up, you need to apply it to an interface for it to start working. [3]

1.5.2 Extended Access List Configuration:

Setting up an extended ACL involves more detailed commands, because you're controlling more aspects of the traffic. But the process is the same: you define your rules with commands, and then you apply your ACL to an interface. If, for example, you wanted to block a certain kind of traffic (like HTTP) from a specific place to a specific destination, you'd write commands that say exactly that.

2. Procedure and Discussion

- Building the topology

The topology was built as shown in Figure 2.1. We copied and pasted them to use in each part.

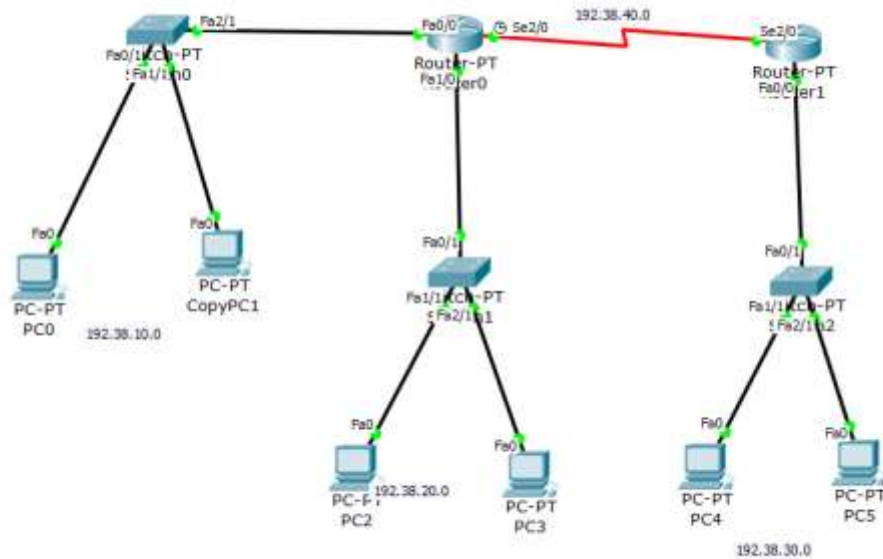


Figure 2.1: The Topology

- Configure the IPs

I defined each device (routers & PCs) IPs by clicking on it and typing in the interfaces. As the figure2.2 below which belongs to PC0.

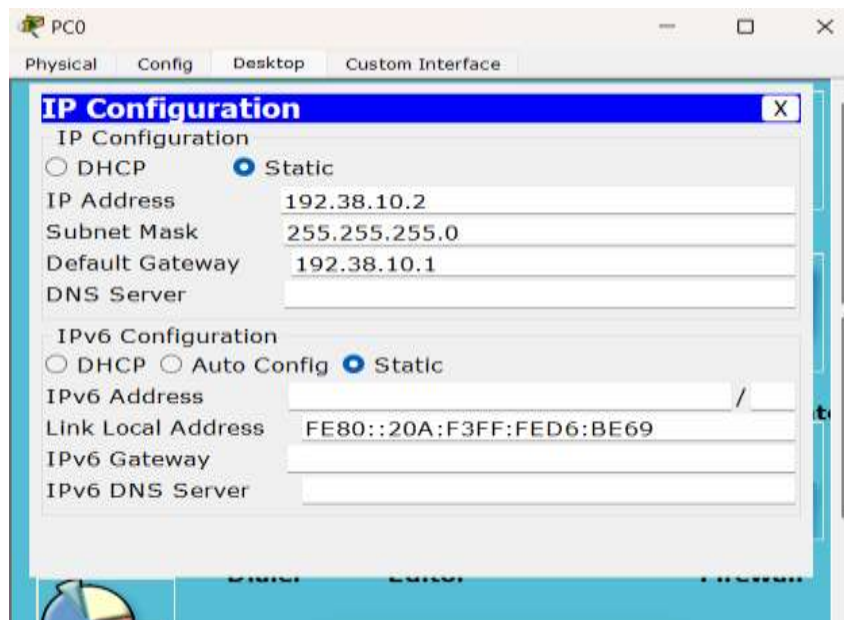


Figure 2.2: PC0 IPs

- Networks IPS

Table 2.1 below show all IPs, Subnet Mask and Wildcard Mas for every device in the topology.

Table 2.1: Networks IPS

Area	Network	Device	Interface	IP	Subnet Mask	Wildcard Mask
Area 0	192.38.40.0/24	Router 0	Se2/0	192.38.40.1	255.255.255.0	0.0.0.255
		Router 1	Se2/0	192.38.40.2	255.255.255.0	0.0.0.255
	192.38.10.0/24	Router 0	Fa0/0	192.38.10.1	255.255.255.0	0.0.0.255
		PC0	Fa0	192.38.10.2	255.255.255.0	0.0.0.255
		PC1	Fa0	192.38.10.3	255.255.255.0	0.0.0.255
	192.38.20.0/24	Router 0	Fa1/0	192.38.20.1	255.255.255.0	0.0.0.255
		PC2	Fa0	192.38.20.2	255.255.255.0	0.0.0.255
		PC3	Fa0	192.38.20.3	255.255.255.0	0.0.0.255
	192.38.30. 0/24	Router 1	Fa0/0	192.38.30. 1	255.255.255.0	0.0.0.255
		PC4	Fa0	192.38.30. 2	255.255.255.0	0.0.0.255
		PC5	Fa0	192.38.30. 3	255.255.255.0	0.0.0.255

2.1. Configuring OSPF Routing

In this section we discuss the OSPF routing protocol for both routers 0 and 1. To be the result for it that all PCs must ping with any point included in the network.

The commands for configuring the OSPF:

➤ router 1

Router(config) #router ospf 1

Router(config-router) #network 192.38.20.0 0.0.0.255 area 0

Router(config-router) #network 192.38.10.0 0.0.0.255 area 0

Router(config-router) #network 192.38.40.0 0.0.0.255 area 0

```
router ospf 1
log-adjacency-changes
network 192.38.20.0 0.0.0.255 area 0
network 192.38.10.0 0.0.0.255 area 0
network 192.38.40.0 0.0.0.255 area 0
```

Figure 2.1: Configuration OSPF Router1

2.2. Configuring Standard Access List

Another idea made in the lab was involved defining rules to allow or deny specific traffic and applying them to interfaces for effective network control.

(Standard ACL takes IDs of 1- 99)

2.2.1. Prevent PC0 to access network 192.38.20.0/24

We first witness the establishment of a standard access list, denoted as 10. Subsequently, this access list is affixed to an interface associated with the source list. Precisely, the interface chosen is Fa0/1, serving as an input interface on the router responsible for receiving packets from the host PC0. It's essential to highlight that selecting Fa0/0 would necessitate defining it as an output interface.

Router (config) #access-list 10 deny host 192.38.10.2

Router (config)#access-list 10 permit any

Router (config) #interface fa1/0

Router (config-if) #ip access-group 10 out

```
Router(config)#access-list 10 deny host 192.38.10.2
Router(config)#access-list 10 permit any
Router(config)#interface fa1/0
Router(config-if)#ip access-group 10 out
Router(config-if)#
```

Figure 2.2.1: Prevent PC0 to access 192.38.20.0 network command

2.2.2. Allow just PC0 to access network 192.38.20.0/24 using the Slandered ACLs and deny any other traffic.

Following the execution of the commands in figure 2.2.2, the existing access list is removed, and a new list is crafted to exclusively allow PC0 access to the 192.38.20.0 network. Notably, the command "access-list 20 permit any" is deliberately omitted, recognizing its unnecessary role in the functioning of other devices.

Router (config) #access-list 20 permit host 192.38.10.2

Router(config) #access-list 20 deny any

Router (config) #interface fa1/0

Router (config-if) #ip access-group 20 out

```
Router#conf t
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#access-list 20 permit host 192.38.10.2
Router(config)#access-list 20 deny any
Router(config)#interface fa1/0
Router(config-if)#ip access-group 20 out
```

Figure 2.2.2: permit PC0 to access 192.38.20.0 network command

2.2.3. Prevent network 192.38.10.0 from accessing network 192.38.20.0 only.

To restrict access from the 192.38.10.0 network to only the 192.38.20.0 network, with the use of wildcard options instead of 'any,' the commands as shown in figure 2.2.3 were implemented.

Router (config) #access-list 30 deny 192.38.10.0 0.0.0.255

Router (config) #access-list 30 permit any

Router (config) #interface fal/0

Router (config-if) #ip access-group 30 out

```
Router(config)#access-list 30 deny 192.38.10.0 0.0.0.255
Router(config)#access-list 30 permit any
Router(config)#interface fal/0
Router(config-if)#ip access-group 30 out
```

Figure 2.2.3: deny network 192.38.10.0 to access network 192.38.20.0 command

2.2.4. Prevent PC0 from accessing Network 192.38.30.0 all other traffic is allowed.

Router (config) #access-list 40 deny host 192.38.10.2

Router (config) #access-list 40 permit any

Router (config) #interface fa0/0

Router (config-if) #ip access-group 40 out

```
Router(config)#access-list 40 deny host 192.38.10.2
Router(config)#access-list 40 permit any
Router(config)#interface fa0/0
Router(config-if)#ip access-group 40 out
Router(config-if)#
```

Figure 2.2.4: Prevent PC0 to access network 192.38.30.0 command

We wrote the above command at figure 2.2.4 using Router 2 that the correct router to setup in order to allow the required prevention. This is due to the fact that extended access lists only evaluate the source address. If we setup Router 1 instead, PC0 will be unable to access other networks, such as the 192.46.20.0 network, so we repeat the previous procedures.

2.3. Configuring Extended Access List

In this part we discussed extended access list identified by the range of IDs from 100 to 199.

2.3.1. Prevent PC0 from accessing PC2. (all other traffic is allowed)

In this section we specified a rule to deny all IP traffic from 192.38.10.2 to 192.38.20.2. This effectively prevents PC0 from accessing PC2 while allowing all other types of traffic

Router (config) #access-list 101 deny ip host 192.38.10.2 host 192.38.20.2

Router (config) #access-list 101 permit ip any any

Router (config) #interface fa0/0

Router (config-if) #ip access-group 101 in

2.3.2. Allow PC0 to access PC2 all other traffic is denied.

Router (config) #access-list 102 permit ip host 192.38.10.2 host 192.38.20.2

Router (config) #access-list 102 deny ip any any

Router (config) #interface fa0/0

Router (config-if) #ip access-group 102 in

```
Router(config)#access-list 102 permit ip host 192.38.10.2 host 192.38.20.2
Router(config)#access-list 102 deny ip any any
Router(config)#interface fa0/0
Router(config-if)#ip access-group 102 in
Router(config-if)#
```

Figure 2.3.2: Allow PC0 to access PC2 all other traffic is denied(command)

2.3.3. Add a server to the topology to network 192.38.20.0/24 and activate http service on it, then deny PC0 to make HTTP request to this server.

After integrated a server into the topology, enabled the HTTP service on the server, then implemented a configuration to prevent PC0 from initiating HTTP requests to this added server.

Router (config) #access-list 103 deny tcp host 192.38.10.2 host 192.38.20.4 eq 80

Router (config) #access-list 103 permit ip any any

Router (config) #interface fa0/0

Router (config-if) #ip access-group 103 in

```
Router(config)#access-list 103 deny tcp host 192.38.10.2 host 192.38.20.4 eq 80
Router(config)#access-list 103 permit ip any any
                                     ^
% Invalid input detected at '^' marker.

Router(config)#access-list 103 permit ip any any
Router(config)#interface fa0/0
Router(config-if)#ip access-group 103 in
```

Figure 2.3.3: deny PC0 to make HTTP request to this server command

2.3.4. Prevent PC0 from accessing PC4 all other traffic is allowed.

Restrict access from PC0 to PC4 while allowing all other types of traffic. The following commands achieve this goal:

Router (config) #access-list 106 deny ip host 192.38.10.2 host 192.38.30.2

Router (config) #access-list 106 permit ip any any

Router (config) #interface fa0/0

Router (config-if) #ip access-group 106 in

```
Router(config)#access-list 106 deny ip host 192.38.10.2 host 192.38.30.2
Router(config)#access-list 106 permit ip any any
Router(config)#interface fa0/0
Router(config-if)#ip access-group 106 in
```

Figure 2.3.4: Prevent PC0 from accessing PC4 command

2.3.5. Enable telnet on Router1 then, deny all the host from making telnet with interface se2/0 of Router1 expect PC0, it can make telnet with any interface.

After enabled Telnet on Router1, we set an extended access list to restrict all hosts, save PC0, from making Telnet sessions to Router1's Se2/0 interface. This guarantees that PC0 retains the ability to create Telnet connections across any interface while preventing other hosts from accessing Telnet over Router1's interface Se2/0.

Router (config) #access-list 107 permit tcp host 192.38.10.2 any eq telnet

Router (config) #access-list 107 deny tcp any any eq telnet

Router (config) #access-list 107 permit ip any any

Router (config) #interface se2/0

Router (config-if) #ip access-group 107 in

```
Router(config)#access-list 107 permit tcp host 192.38.10.2 any eq telnet
Router(config)#access-list 107 deny tcp any any eq telnet
Router(config)#access-list 107 permit ip any any
Router(config)#interface se2/0
Router(config-if)#ip access-group 107 in
Router(config-if)#
```

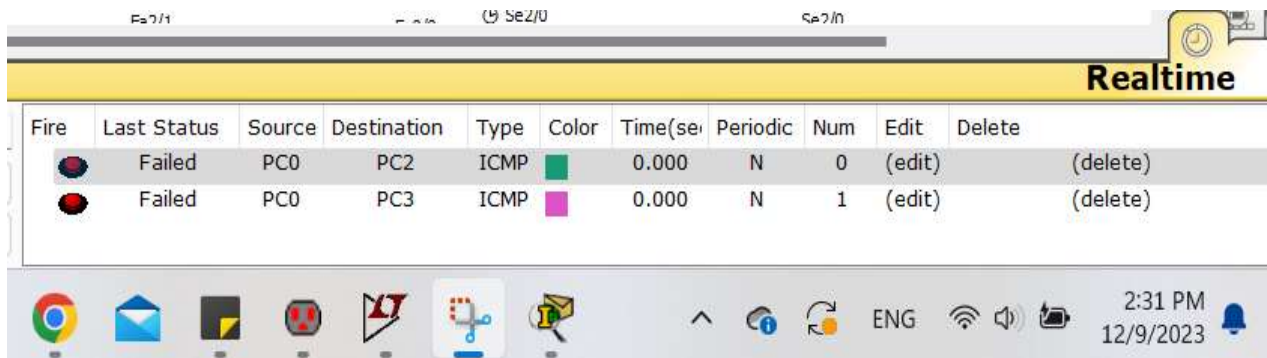
Figure 2.3.5: deny all the host from making telnet with interface se2/0 of Router1 expect PC0(command)

3. Results

3.1. Configuring Standard Access List

3.1.1. Prevent PC0 to access network 192.38.20.0/24

The result of this configuration is shown in figure3.1.1 below, which shows that PC0 is effectively prevented from accessing any hosts on the 192.38.20.0 network.



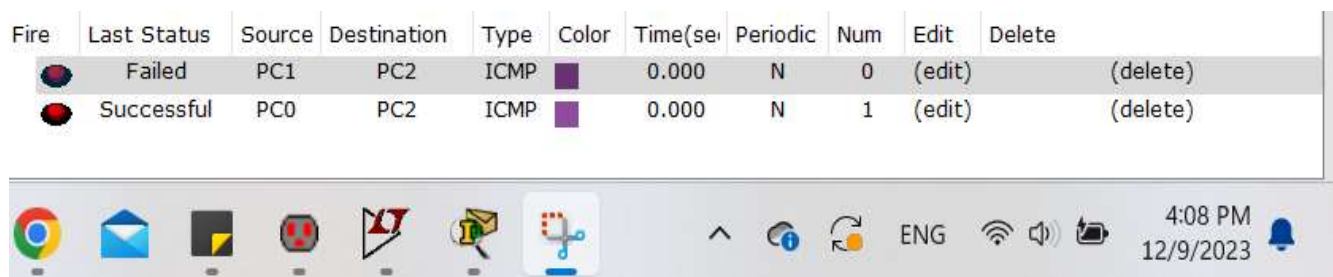
The screenshot shows a 'Realtime' monitoring window with a table of network events. The table has columns: Fire, Last Status, Source, Destination, Type, Color, Time(se), Periodic, Num, Edit, and Delete. Two rows are visible, both with a status of 'Failed' and source 'PC0'. The first row shows a failed ICMP attempt to PC2 with a green color indicator. The second row shows a failed ICMP attempt to PC3 with a pink color indicator. The interface includes a taskbar at the bottom with various application icons and system status information like 'ENG', signal strength, and the date '12/9/2023'.

Fire	Last Status	Source	Destination	Type	Color	Time(se)	Periodic	Num	Edit	Delete
	Failed	PC0	PC2	ICMP	Green	0.000	N	0	(edit)	(delete)
	Failed	PC0	PC3	ICMP	Pink	0.000	N	1	(edit)	(delete)

Figure 3.1.1: Prevent PC0 to access 192.38.20.0 network output

3.1.2. Allow just PC0 to access network 192.38.20.0/24 using the Slandered ACLs and deny any other traffic.

The objective here is to establish a Standard ACL configuration that permits solely PC0 to access the specified network while denying any other form of traffic. As the result in figure 3.1.2.



The screenshot shows a 'Realtime' monitoring window with a table of network events. The table has columns: Fire, Last Status, Source, Destination, Type, Color, Time(se), Periodic, Num, Edit, and Delete. Two rows are visible. The first row shows a failed ICMP attempt from PC1 to PC2 with a purple color indicator. The second row shows a successful ICMP attempt from PC0 to PC2 with a purple color indicator. The interface includes a taskbar at the bottom with various application icons and system status information like 'ENG', signal strength, and the date '12/9/2023'.

Fire	Last Status	Source	Destination	Type	Color	Time(se)	Periodic	Num	Edit	Delete
	Failed	PC1	PC2	ICMP	Purple	0.000	N	0	(edit)	(delete)
	Successful	PC0	PC2	ICMP	Purple	0.000	N	1	(edit)	(delete)

Figure 3.1.2: permit PC0 to access 192.38.20.0 network output

That is what it required because just PC0 able to access network 192.38.20.0 and the others unable to access like PC1.

3.1.3. Prevent network 192.38.10.0 from accessing network 192.38.20.0 only.

The results that achieved this appear in figure 3.1.3. Which is explained by try access from any device in network 192.38.10.0 (PC0,PC1) to network 192.38.20.0 (PC2) but it failed.

Fire	Last Status	Source	Destination	Type	Color	Time(se	Periodic	Num	Edit	Delete
	Failed	PC1	PC2	ICMP		0.000	N	0	(edit)	(delete)
	Failed	PC0	PC2	ICMP		0.000	N	1	(edit)	(delete)

Figure 3.1.3: deny network 192.38.10.0 to access network 192.38.20.0 output

3.1.4. Prevent PC0 from accessing Network 192.38.30.0 all other traffic is allowed.

Fire	Last Status	Source	Destination	Type	Color	Time(se	Periodic	Num	Edit	Delete
	Failed	PC0	PC4	ICMP		0.000	N	0	(edit)	(delete)
	Failed	PC0	PC5	ICMP		0.000	N	1	(edit)	(delete)

Figure 3.1.4: Prevent PC0 to access network 192.38.30.0 output

Its result in figure 3.1.4 showed that PC0 cannot access with the devices in network 192.38.30.0 (PC4 , PC5).

3.2. Configuring Extended Access List

3.2.1. Prevent PC0 from accessing PC2. (all other traffic is allowed)

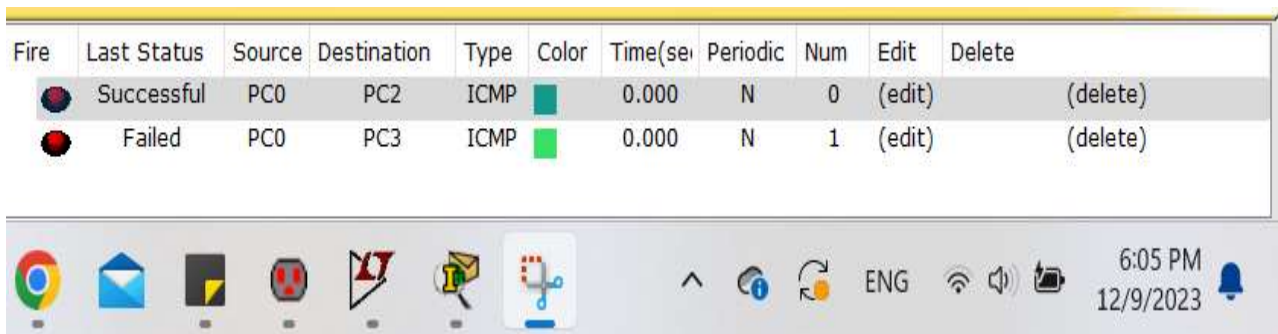
After run its commands, the output was appeared as shown in figure 3.2.1 that PC0 cannot access with PC2 but can access with PC3 which is in PC0 network.

Fire	Last Status	Source	Destination	Type	Color	Time(se	Periodic	Num	Edit	Delete
	Successful	PC0	PC3	ICMP		0.000	N	0	(edit)	(delete)
	Failed	PC0	PC2	ICMP		0.000	N	1	(edit)	(delete)

Figure 3.2.1: Prevent PC0 from accessing PC2 output

3.2.2. Allow PC0 to access PC2 all other traffic is denied.

When we run the commands in figure 2.3.2, we had this result in figure 2.3.2.1 so the test is success that the access from PC0 to PC2 is failed.



Fire	Last Status	Source	Destination	Type	Color	Time(se)	Periodic	Num	Edit	Delete
	Successful	PC0	PC2	ICMP		0.000	N	0	(edit)	(delete)
	Failed	PC0	PC3	ICMP		0.000	N	1	(edit)	(delete)

Figure 3.2.2: Allow PC0 to access PC2 all other traffic is denied(output)

3.2.3. Add a server to the topology to network 192.38.20.0/24 and activate http service on it, then deny PC0 to make HTTP request to this server.

Upon execution of the provided commands, the anticipated outcomes have been successfully realized, as depicted in the detailed results presented in Figure 3.2.3 that confirmed the effective prevention of PC0 from making HTTP requests to the newly added server.

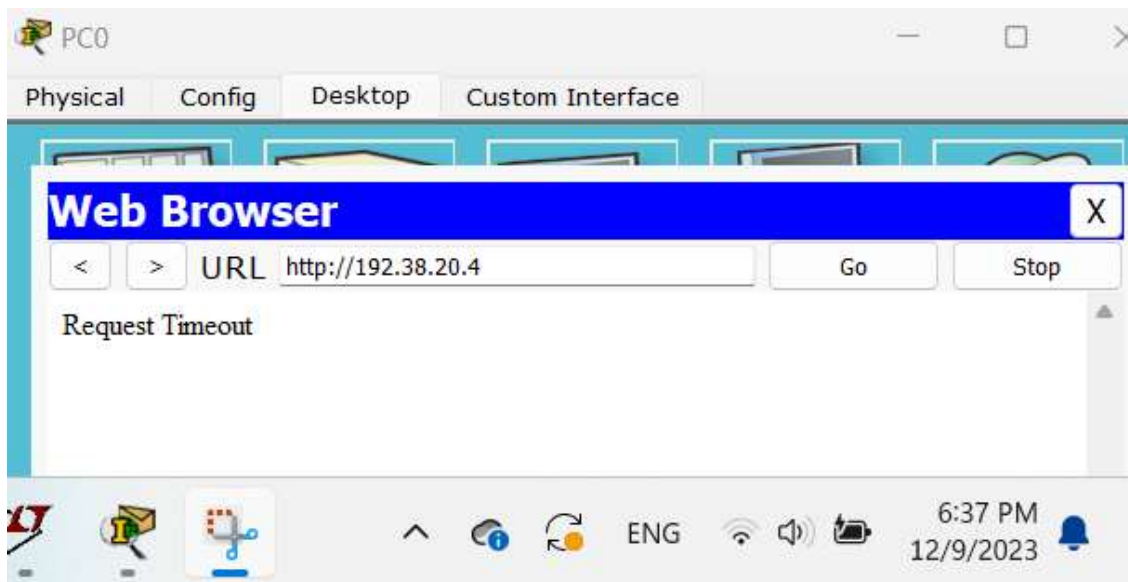
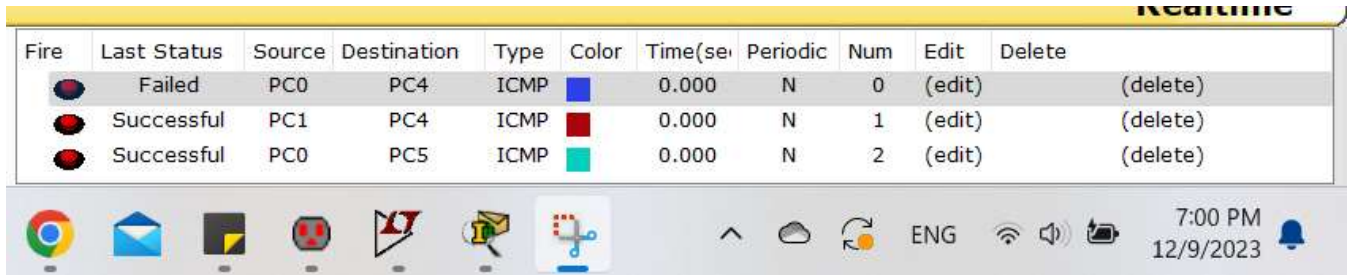


Figure 3.2.3: deny PC0 to make HTTP request to this server output

3.2.4. Prevent PC0 from accessing PC4 all other traffic is allowed.

After run the commands we got the result that we want in figure 3.2.4, which PC0 failed to access PC4 and the other devices is allowed as PC1 access PC0 successfully.

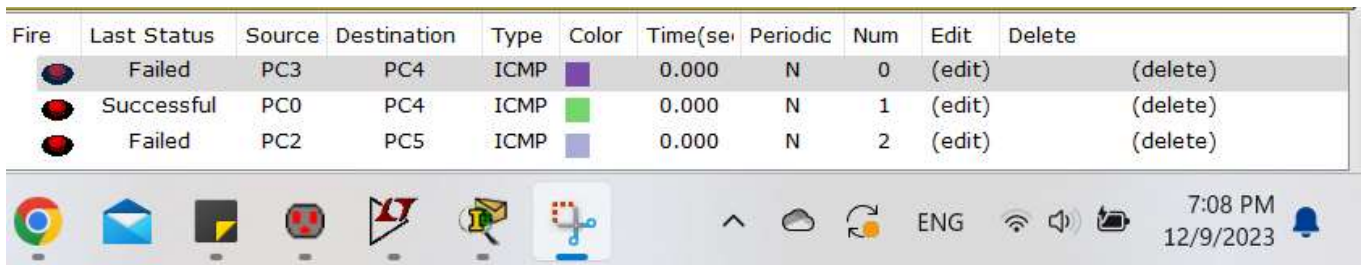


Fire	Last Status	Source	Destination	Type	Color	Time(se)	Periodic	Num	Edit	Delete
	Failed	PC0	PC4	ICMP	Blue	0.000	N	0	(edit)	(delete)
	Successful	PC1	PC4	ICMP	Red	0.000	N	1	(edit)	(delete)
	Successful	PC0	PC5	ICMP	Green	0.000	N	2	(edit)	(delete)

Figure 3.2.4: Prevent PC0 from accessing PC4 output

3.2.5. Enable telnet on Router1 then, deny all the host from making telnet with interface se2/0 of Router1 expect PC0, it can make telnet with any interface.

The output in figure 3.2.5 showed that access from PC0 to PC4 was successful, but attempts from PC3 to PC4 and PC2 to PC5 were failed, showing the correctness of access control policies in allowing selective communication.



Fire	Last Status	Source	Destination	Type	Color	Time(se)	Periodic	Num	Edit	Delete
	Failed	PC3	PC4	ICMP	Purple	0.000	N	0	(edit)	(delete)
	Successful	PC0	PC4	ICMP	Green	0.000	N	1	(edit)	(delete)
	Failed	PC2	PC5	ICMP	Blue	0.000	N	2	(edit)	(delete)

Figure 3.2.5: deny all the host from making telnet with interface se2/0 of Router1 expect PC0(output)

Conclusion

We studied OSPF routing, Standard Access Lists (ACLs), and Extended ACLs in this project. OSPF was set up for effective network routing. We define rules in Standard ACLs to limit PC0's access to certain networks, permitting or refusing traffic as appropriate. Extended ACLs were then used to fine-tune access restrictions, prohibiting or permitting particular host communication. Hands-on work included setting Telnet limits on Router1 and restricting HTTP access for PC0. Overall, the exercise improved our hands-on experience with network configuration and security measures.

6. References

[1] Link: https://www.cisco.com/E-Learning/prod/curriculum/cco_tdo_1dd/demos/DandEguideFRE/D3ChallengeACLlab.pdf

[Accesses Time: 9/12/2023 8:30pm]

[2] Link: <https://www.youtube.com/watch?v=c7vywj895tA>

[Accesses Time: 9/12/2023 8:34pm]

[3] Link: <https://www.cisco.com/c/en/us/support/docs/security/ios-firewall/23602-confaccesslists.html>

[Accesses Time: 12/12/2023 11:19pm]